P0019r00 : Atomic View

Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: ogiroux@nvidia.com
Author: James Reus
Contact: reus1@llnl.gov

Number: P0019
Version: 00
Date: 2015-09-23
URL: https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0019.rst
WG21: SG1 Concurrency

1 Introduction

This paper proposes an extension to the atomic operations library [atomics] for atomic operations applied to non-atomic objects. The proposal is in five parts: (1) the concept of an atomic view, (2) application of this concept applied to single objects, (3) application of this concept applied to members of a very large array, (4) application of this concept applied to an object in a large legacy code which cannot replace that object with a corresponding atomic<T> object, and (5) motivating use cases and illustrative examples.

```
namespace std {
namespace experimental {
  template< class T > atomic_view ;
  template< class T > atomic_array_view ;
  template< class T > atomic_global_view ;
}}
```

1.1 Set of Transitive Copies and its Originating Instance

Definition: **Set of Transitive Copies**
An instance 'b' of type T is a **transitive copy** of an instance 'a' of the same type if-and-only-if 'b' is copy constructed from 'a', assigned from 'a', copy constructed from a transitive copy of 'a', or assigned from a transitive copy of 'a'. An instance is removed from its *set of transitive copies* when it is destroyed, reset to an unassigned state, or assigned into a different *set of transitive copies*.

Definition: **Originating Instance**
A *set of transitive copies* has an **originating instance** from which all other members are transitively copy constructed or assigned.

For example, instances of **std::shared_ptr<T>** that share ownership of a given object form a set of transitive copies.

1.2 Applicability to Atomic View

Operations provided through an **atomic_view<T>** instance are guaranteed atomic [1.10, Multi-threaded executions and data races] only with respect to operations performed through members of the *set of transitive copies* of **atomic_view<T>** to which the instance belongs.

Operations provided through an **atomic_array_view<T>** instance are guaranteed atomic [1.10, Multi-threaded executions and data races] only with respect to operations performed through members of the *set of transitive copies* of atomic_array_view<T> to which the instance belongs.

2 Atomic View Concept

A class conforming to the atomic view concept provides atomic operations for the viewed non-atomic object. These operations replicate the operations on **std::atomic<T>** [29.5, Atomic types]; however, operations are const with respect to the atomic view object in contrast to the non-const and volatile operations of a **std::atomic<T>** object. An atomic view does not own (neither exclusively nor shared) the viewed non-atomic object.

The following *atomic-view-concept* specification is included to define requirements for classes conforming to the *atomic-view-concept* and does not imply the existiance of a template class of this name.

```
template< class T >
struct atomic-view-concept {
    // Functionality matches corresponding member of atomic<T>
    bool is_lock_free() const noexcept;
    void store( T , memory_order = memory_order_seq_cst ) const noexcept;
    T load( memory_order = memory_order_seq_cst ) const noexcept;
    operator T() const noexcept ;
    T exchange( T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( T&, T, memory_order = memory_order_seq_cst ) const noexcept;
    T operator=(T) const noexcept ;

    // Functionality that may exist and deviate from corresponding member of atomic<T>, if any.
    atomic-view-concept ();
    atomic-view-concept ( const atomic-view-concept & );
    atomic-view-concept ( atomic-view-concept && );
    atomic-view-concept & operator = ( const atomic-view-concept & );
    atomic-view-concept & operator = ( atomic-view-concept && );

    constexpr explicit bool operator() const noexcept;
};
```

Constructors and assignment operators of an *atomic-view-concept* may acquire resources such as concurrent locks to support atomic operations on the non-atomic object, and may track membership in a **set of transitive copies** for the purpose of sharing those resources.

constexpr explict bool operator() const noexept ;

Returns if the *atomic-view-concept* object views an object. A default constructed *atomic-view-concept* object returns false.

A class conforming to the atomic view concept shall provide the following operations when T is an integral type. These operations replicate the operations on *std::atomic<integral>* [29.5, Atomic types]; however, operations are const with respect to the atomic view object in contrast to the non-const and volatile operations of a **std::atomic<integral>** object.

```
template<> struct atomic-view-concept < integral > {

    integral fetch_add( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_sub( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_and( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_or( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_xor( integral , memory_order = memory_order_seq_cst) const noexcept;

    integral operator++(int) const noexcept;
    integral operator--(int) const noexcept;
    integral operator++() const noexcept;
    integral operator--() const noexcept;
    integral operator+=( integral ) const noexcept;
    integral operator-=( integral ) const noexcept;
    integral operator&=( integral ) const noexcept;
    integral operator|=( integral ) const noexcept;
    integral operator^=( integral ) const noexcept;
};
```

Note that for consistency the *atomic-view-concept<integral>* mathematical operator overloads retain the same mathematical inconsistency with respect to the mathematical operators for the *integral* type, as illustrated below.

```
int i(0)
++( ++i );        // ++i returns an lvalue
( i += 1 ) += 2 ; // i+= returns an lvalue

std::atomic<int> ai(0);
++( ++( ai ) );    // error: ++ai returns an rvalue
( ai += 1 ) += 2 ; // error: ai+= returns an rvalue
```

3 Atomic View for a Single Object

An **atomic_view<T>** object is used to perform atomic operations on the viewed non-atomic object. The intent is for **atomic_view<T>** to provide the best-performing implementation of *atomic-view-concept* operations for the type T.

```
template< class T > struct atomic_view { // conforms to atomic view concept

    explicit atomic_view( T & ); // Originating Constructor is NOT noexcept

    atomic_view();
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    ~atomic_view() noexcept ;
};
```

[Note: The intent is for atomic operations of *atomic_view<T>* to directly update the referenced object. The set of transitive copies of *atomic_view<T>* may require a resource, such as a locking mechanism, to perform atomic operations. The intent is to enable amortization of the time and space overhead of obtaining and releasing such a resource. – end note]

atomic_view<T>::atomic_view(T & obj);

Requires: The referenced obj must be properly aligned for its type T, otherwise behavior is undefined.

Effects: This originating constructor wraps the referenced object. The constructed instance is the originating member of a **set of transitive copies** of **atomic_view<T>**. [Note: This constructor may obtain a resource as necessary to support atomic operations. The originating constructor is allowed to throw an exception if such a resource could not be obtained. – end note]

atomic_view<T>::atomic_view(const atomic_view & rhs) noexcept ;

Effects: If rhs is a member of a set of transitive copies of atomic_view<T> the copy constructed instance is a member of that set.

atomic_view<T>::~atomic_view() noexcept ;

Effects: If this instance is a member of a *set of transitive copies* then this instance is removed from the set. [Note: If the set will become empty then a resource shared by that set should be released. – end note]

atomic_view<T> & atomic_view<T>::operator = (const atomic_view & rhs) noexcept ;

Effects: If this instance is a member of a *set of transitive copies* then that instance is removed from the set. [Note: If the set will become empty then a resource shared by that set should be released. – end note] If rhs is a member of a set of transitive copies of **atomic_view<T>** the copy constructed instance is a member of that set.

4 Atomic View for a Very Large Array

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An **atomic_array_view<T>** object is used to perform atomic operations on the viewed non-atomic members of the array. The intent is for **atomic_array_view<T>** to provide the best-performing implementation of atomic-view-concept operations for the members of the array.

```
template< class T > struct atomic_array_view {

    bool is_lock_free() const noexcept ;

    // Returns true if the view wraps an array and member access is valid.
    explicit bool operator() const noexcept ;

    atomic_array_view( T * , size_t ); // Originating Constructor is NOT noexcept
    atomic_array_view() noexcept ;
    atomic_array_view( atomic_array_view && ) noexcept ;
    atomic_array_view( const atomic_array_view & ) noexcept ;
    atomic_array_view & operator = ( const atomic_array_view & ) noexcept ;
    ~atomic_array_view() noexcept ;

    size_t size() const noexcept ;

    typedef implementation-defined-atomic-view-concept-type reference ;

    reference operator[]( size_t ) const noexcept ;
};
```

[Note: The intent is for atomic operations on members of **atomic_array_view<T>** to directly update the referenced member. The *set of transitive copies* of **atomic_array_view<T>** may require resources, such as locking mechanisms, to perform atomic operations. The intent is to enable amortization of the time and space overhead of obtaining and releasing such resources. – end note]

typedef *implementation-defined-atomic-view-concept-type* **reference;**

The **reference** type conforms to *atomic-view-concept* for type T.

bool atomic_array_view<T>::is_lock_free() const noexcept ;

Effects: Returns whether atomic operations on members are lock free.

atomic_array_view<T>::atomic_array_view(T * ptr , size_t N);

Requires: The array referenced by [ptr .. ptr+N-1] must be properly aligned for its type T, otherwise behavior is undefined.

Effects: This *originating constructor* wraps the referenced array [ptr .. ptr+N-1]. The constructed instance is the originating member of a *set of transitive copies* of atomic_array_view<T>. [Note: This constructor may obtain resources as necessary to support atomic operations. The originating constructor is allowed to throw an exception if such resources could not be obtained. – end note]

atomic_array_view<T>::atomic_array_view(const atomic_array_view & rhs) noexcept ;

Effects: If rhs is a member of a set of transitive copies of atomic_array_view<T> the copy constructed instance is a member of that set.

atomic_array_view<T>::~atomic_array_view() noexcept ;

Effects: If this instance is a member of a set of transitive copies this instance is removed from the set. [Note: If the set will become empty then resources shared by that set should be released. – end note]

atomic_array_view<T> & atomic_array_view<T>::operator = (const atomic_array_view & rhs) noexcept ;

Effects: If this instance is a member of a set of transitive copies that instance is removed from the set. [Note: If the set will become empty then resources shared by that set should be released. – end note] If rhs is a member of a set of transitive copies of atomic_array_view<T> the copy constructed instance is a member of that set.

atomic_array_view<T>::reference atomic_array_view<T>::operator[](size_t i) const noexcept ;

Requires: The index i must be in the range [0 .. N-1], otherwise behavior is undefined.

Effects: Return an instance of **reference** type for the member object referenced by the input index i. [Note: The intent is for efficient generation of the returned instance with respect to obtaining a resource, such as a shared locking mechanism, that may be required to support atomic operations on the referenced member. – end note]

5 Atomic Global Views for a Single Non-atomic Object

An **atomic_global_view<T>** object is used to perform atomic operations on the globally accessible viewed non-atomic object. The intent is for **atomic_global_view<T>** to provide the best-performing implementation of *atomic-view-concept* operations for the type T. All atomic operations on an instance of **atomic_global_view<T>** are atomic with respect to any other instance that views the same globally accessible object, as defined by equality of pointers to that object.

[Note: Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations. Such replacement may not be able to introduce a set of transitive copies of atomic_view<T>. – end note]

```
template< class T > struct atomic_global_view { // conforms to atomic view concept

    atomic_global_view( T & ); // Wrapping constructor is NOT noexcept
    atomic_global_view( const atomic_global_view & ) noexcept ;
    atomic_global_view( atomic_global_view && ) noexcept ;
    ~atomic_global_view() noexcept ;

    atomic_global_view() = delete ;
    atomic_global_view & operator = ( const atomic_concurrent__view & ) = delete ;
};
```

[Note: The intent is for atomic operations of **atomic_global_view<T>** to directly update the referenced object. – end note]

atomic_global_view<T>::atomic_global_view(T & obj);

Requires: The referenced obj must be properly aligned for its type T, otherwise behavior is undefined.

Effects: This wrapping constructor wraps the globally accessible referenced object. Atomic operations on this instance are atomic with respect to atomic operations on any **atomic_global_view<T>** instance that reference the same globally accessible object. [Note: This constructor may obtain a resource as necessary to support atomic operations. This constructor is allowed to throw an exception if such a resource could not be obtained. – end note]

atomic_global_view<T>::atomic_global_view(const atomic_global_view &) noexcept ;
atomic_global_view<T> & atomic_global_view<T>::operator = (const atomic_global_view &) noexcept ;

Effects: If rhs references a globally accessible object then this instance references the same object otherwise this instance does not reference a globally accessible object.

atomic_global_view<T>::~atomic_global_view() noexcept ;

Effects: This instance does not reference a globally accessible object.

6 Notes and Examples

6.1 Atomic View

All non-atomic accesses of the wrapped object that appear before the wrapping constructor must happen before subsequent atomic operations on the atomic_view. For example:

```
void foo( int & i ) {
  i = 42 ;
  atomic_view<int> ai(i);
  // Operations on ‘i’ must happen before operations on ‘ai’
  foreach( parallel_policy, 0, N, [=](){ ++ai ; } );
}
```

6.2 Atomic Array View

Under the HPC use case the member access operator, proxy type constructor, or proxy type destructor will be frequently invoked; therefore, an implementation should trade off decreased overhead in these operations versus increased overhead in

the wrapper constructor and final destructor.

Usage Scenario for **atomic_array_view<T>**

a. A very large array of trivially copyable members is allocated.
b. A parallel algorithm initializes members through non-conflicting assignments.
c. The array is wrapped by an atomic_array_view<T>.
d. One or more parallel algorithms update members of the array through atomic view operations.
e. The atomic_array_view<T> is destructed.
f. Parallel algorithms access array members through non-conflicting reads, writes, or updates.

Example:

```
// atomic array view wrapper constructor:
atomic_array_view<T> array( ptr , N );

// atomic operation on a member:
array[i].atomic-operation(...);

// atomic operations through a temporary value
// within a concurrent function:
atomic_array_view<T>::reference x = array[i];
x.atomic-operation-a(...);
x.atomic-operation-b(...);
```

Possible interface for **atomic_array_view<T>::reference**

```
struct implementation-defined-proxy-type {   // conforms to atomic view concept

  // Construction limited to move
  implementation-defined-proxy-type(implementation-defined-proxy-type && ) = noexcept ;
  ~implementation-defined-proxy-type();

  implementation-defined-proxy-type() = delete ;
  implementation-defined-proxy-type( const implementation-defined-proxy-type & ) = delete ;
  implementation-defined-proxy-type &
    operator = ( const implementation-defined-proxy-type & ) = delete ;
};
```

Originating constructor options for **atomic_array_view<T>**

> A originating constructor of the form (T*begin, T*end) could be valid. However, the (T*ptr, size_t N) version is preferred to minimize potential confusion with construction from non-contiguous iterators. Wrapping constructors for standard contiguous containers would also be valid. However, such constructors could have potential confusion as to whether the atomic_array_view would or would not track resizing operations applied to the input container.

Implementation note for **atomic_array_view<T>**

> All non-atomic accesses of array members that appear before the wrapping constructor must happen before subsequent atomic operations on the atomic_array_view members. For example:

```
void foo( int * i , size_t N ) {
  i[0] = 42 ;
  i[N-1] = 42 ;
  atomic_array_view<int> ai(i,N);
  // Operations on 'i' must happen before operations on 'ai'
  foreach( parallel_policy, 0, M, [=]( int j ){ ++ai[j%N] ; } );
}
```

6.3 Atomic Global View

All non-atomic accesses of the wrapped object that appear before the wrapping constructor must happen before subsequent atomic operations on the atomic_view. For example:

```
void foo( int & i ) {
  i = 42 ;
  // Operations on 'i' must happen before operations on 'ai'
  foreach( parallel_policy, 0, N, [=](){ ++atomic_global_view<ai>(i) ; } );
}
```

Example:

```
// atomic operation on an object:
```

```
atomic_global_view<T>(x).atomic-operation(...);

// When multiple atomic operations are performed the cost of
// constructing and destructing the atomic view can be amortized
// through a temporary atomic view object.
{
  atomic_global_view ax(x);
  ax.atomic-operation-a(...);
  ax.atomic-operation-b(...);
}
```

6.4 Mathematically Consistent Integral Operator Overloads

As previously noted the **std::atomic**<*integral*> mathematical operator overloads are inconsistent with the mathematical operators for *integral*. The *atomic-view-concept*<*integral*> retains these inconsistent operator overloads. Consistent mathematical operator semantics would be restored with the following operator specifications. However, such a change would break backward compatibility and is therefore only noted and not a proposed change.

template<> struct atomic < *integral* > {

    volatile atomic & operator++(int) volatile noexcept ;
    atomic & operator++(int) noexcept ;
    volatile atomic & operator--(int) volatile noexcept ;
    atomic & operator--(int) noexcept ;

    // fetch-and-increment, fetch-and-decrement operators:
    integral operator++() volatile noexcept ;
    integral operator++() noexcept ;
    integral operator--() volatile noexcept ;
    integral operator--() noexcept ;

    volatile atomic & operator+=(*integral*) volatile noexcept;
    atomic & operator+=(*integral*) noexcept;
    volatile atomic & operator-=(*integral*) volatile noexcept;
    atomic & operator-=(*integral*) noexcept;
    volatile atomic & operator&=(*integral*) volatile noexcept;
    atomic & operator&=(*integral*) noexcept;
    volatile atomic & operator|=(*integral*) volatile noexcept;
    atomic & operator|=(*integral*) noexcept;
    volatile atomic & operator^=(*integral*) volatile noexcept;
    atomic & operator^=(*integral*) noexcept;
};

template<> struct *atomic-view-concept* < *integral* > {

    const *atomic-view-concept* & operator++(int) const noexcept;
    const *atomic-view-concept* & operator--(int) const noexcept;

    integral operator++() const noexcept;
    integral operator--() const noexcept;

    const *atomic-view-concept* & operator+=(*integral*) const noexcept;
    const *atomic-view-concept* & operator-=(*integral*) const noexcept;
    const *atomic-view-concept* & operator&=(*integral*) const noexcept;
    const *atomic-view-concept* & operator|=(*integral*) const noexcept;
    const *atomic-view-concept* & operator^=(*integral*) const noexcept;
};